

Remediación de Vulnerabilidades OWASP

Nueva rama de git:

```
C:\Users\valec\OneDrive\Documentos\git>git checkout -b secure  
Switched to a new branch 'secure'
```

Antes y después de SQL Injection en ASP.NET Core

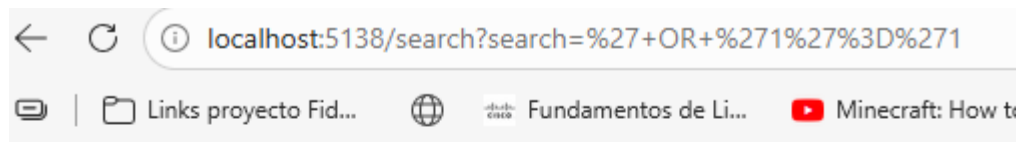
VulnerableApp [Home](#) [Privacy](#)

Buscar Usuarios

Username	Email	Balance
admin	admin@test.com	\$1000.00
user1	user@test.com	\$500.00
user2	user2@test.com	\$750.00

VulnerableApp [Home](#) [Privacy](#)

Buscar Usuarios



VulnerableApp Home Privacy

Buscar Usuarios

Captura de código corregido:

```
15 public IActionResult Index(string search)
16 {
17     if (string.IsNullOrEmpty(search))
18     {
19         return View(new List<User>());
20     }
21     var users = _db.Users
22         .Where(u => u.Username.Contains(search))
23         .ToList();
24     return View(users);
25 }
```

Antes y después de Autenticación

VulnerableApp Home Privacy

Login

VulnerableApp Home Privacy

Dashboard

Usuario: user1

Email: user@test.com

Balance: 500.00



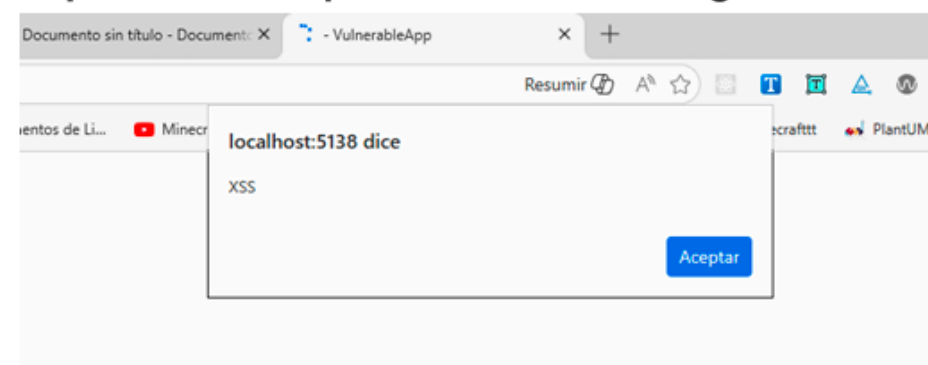
Captura de código corregido:

```
22  
23     var user = _db.Users.FirstOrDefault(u => u.Username == username);  
24  
25     if (user == null || !BCrypt.Net.BCrypt.Verify(password, user.PasswordHash))  
26     {  
27         ViewBag.Error = "Credenciales inválidas"; "Credenciales": Unknown word.  
28         return View();  
29     }  
30  
31     HttpContext.Session.SetString("User", user.Username);  
32     HttpContext.Session.SetInt32("UserId", user.Id);  
33     return RedirectToAction("Dashboard");  
34 }
```

Antes y después de XSS



Captura del comportamiento en navegador:



Comentarios

`<script>alert('XSS')</script>`

Comentar

`<script>alert('XSS')</script>`

Comentarios

Comentar

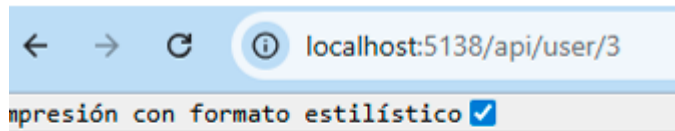
`<script>alert('XSS')</script>`

`<script>alert('XSS')</script>`

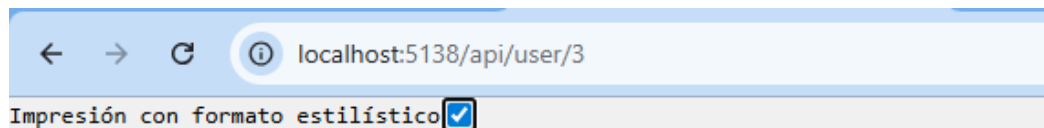
Captura de código corregido:

```
7 <div id="comments">
8   @foreach (var comment in Model ?? new List<string>())
9   {
10    <div class="comment">@Html.Encode(comment)</div>
11  }
12 </div>
```

Antes y después de IDOR

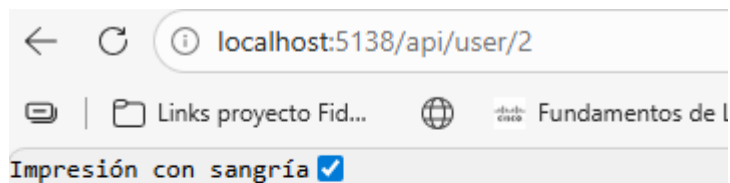


```
"id": 3,  
"username": "user2",  
"email": "user2@test.com",  
"balance": 750,  
"password": "password"
```



```
{  
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.2",  
  "title": "Unauthorized",  
  "status": 401,  
  "traceId": "00-d35afcc509977f18375387e6807e0f01-8946fbb237d6922d-00"  
}
```

Con inicio de sesión:



```
{  
  "id": 2,  
  "username": "user1",  
  "email": "user@test.com"  
}
```

Captura de código corregido:

```
var currentUserId = HttpContext.Session.GetInt32("UserId");  
if (!currentUserId.HasValue) return Unauthorized();  
  
if (id != currentUserId.Value) return Forbid();  
  
var user = _db.Users.Find(id);  
if (user == null) return NotFound();  
return Ok(new  
{  
    user.Id,  
    user.Username,  
    user.Email  
});
```

Tabla comparativa Final:

Vulnerabilidad	Archivo	Código Vulnerable	Código Corregido	Mecanismo de Mitigación
Inyección SQL	Controllers/ SearchController.cs	<pre>string query = "SELECT * FROM Users WHERE Username LIKE '%" + search + "%'"; var users = _db.Users.FromSqlRaw (query);</pre>	<pre>var users = _db.Users.Where(u => u.Username.Contains(search)).ToList();</pre>	La entrada del usuario se separa de la consulta SQL para que la base de datos la interprete únicamente como información y no como parte del código de la consulta.
Autenticación Rota	Controllers/ AuthController.cs Data/ AppDbContext.cs	<pre>string query = "SELECT * FROM Users WHERE Username = '" + username + "' AND Password = '" + password + "'";</pre>	<pre>var user = _db.Users.FirstOrDefault(u => u.Username == username); if (user == null !BCrypt.Verify(password, user.PasswordHash))</pre>	En lugar de guardar las contraseñas tal como las escribe el usuario, el sistema almacena una versión transformada y segura de ellas, lo que protege las credenciales ante accesos no autorizados.

Cross-Site Scripting (XSS)	Views/Comment/Index.cshtml	<pre><div class="comment"> @Html.Raw(comment) </div></pre>	<pre><div class="comment"> @Html.Encode(comment) </div></pre>	Transforma caracteres especiales en texto seguro para evitar que el navegador ejecute código o scripts introducidos por un usuario.
IDOR	Controllers/ApiController.cs	<pre>var user = _db.Users.Find(id); return Ok(new { ... user.Password });</pre>	<pre>if (id != currentUserId.Value) return Forbid(); return Ok(new { user.Id, user.Username, user.Email });</pre>	Antes de devolver información, el sistema comprueba que el recurso solicitado pertenezca al usuario autenticado. También limita los datos enviados en la respuesta para reducir la exposición de información sensible.